

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>D3 + Labels + Cluster LayerSupport</title>
```

```
  <meta charset="utf-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <!-- 1. MODERN LEAFLET CSS -->
```

```
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
```

```
    <link rel="stylesheet" href="./leaflet-mapwithlabels.css" />
```

```
  <!-- 2. MARKERCLUSTER CSS -->
```

```
  <link rel="stylesheet" href="./MarkerCluster.css" />
```

```
  <link rel="stylesheet" href="./MarkerCluster.Default.css" />
```

```
  <style>
```

```
    body { margin: 0; padding: 0; }
```

```
    #map_div { height: 600px; width: 100%; }
```

```
    .info { padding: 6px 8px; font: 14px/16px Arial; background: white; border-radius: 5px; box-shadow: 0 0 15px rgba(0,0,0,0.2); }
```

```
    .legend i { width: 18px; height: 18px; float: left; margin-right: 8px; opacity: 0.7; }
```

```
    html { height: 100% }
```

```
    body { height: 100%; margin: 0; padding: 0 }
```

```
    #map-canvas { height: 100% }
```

```
    html, body, #map-canvas { height: 100%; margin: 0; padding: 0; background: #f0f0f0; }
```

```
    path.neighborhood {
```

```
      cursor: pointer;
```

transition: fill-opacity 0.3s ease;

}

/* High-visibility label styling - EXTRA BOLD */

.d3-label {

font-family: "Helvetica Neue", Helvetica, Arial, sans-serif;

font-weight: 900;

fill: #111;

text-anchor: middle;

pointer-events: none;

text-shadow: 2px 2px 0px #fff, -2px -2px 0px #fff, 2px -2px 0px #fff, -2px 2px 0px #fff;

}

html, body, #map_div {

height: 100% !important;

width: 100% !important;

margin: 0;

padding: 0;

-webkit-backface-visibility: hidden; /* Fixes "shimmer" on some browsers */

}

/* Add this to your map page styles */

.leaflet-zoom-animated {

display: block !important; /* Forces the SVG layer to stay rendered */

}

.leaflet-tile-container {

transition: opacity 0.2s linear; /* Makes tile loading less jarring */

}

```
/* Ensure the labels and polygons stay on the same rendering plane */
```

```
.leaflet-pane {  
  z-index: 400;  
}
```

```
/* Prevents the SVG renderer from clipping polygons during a resize */
```

```
/* Force the SVG container to always fill the available space */
```

```
/* Force the SVG container to fill the 100% space of the expanded iframe */
```

```
.s{  
  width: 100% !important;  
  height: 100% !important;  
  transform-origin: 0 0 !important;  
  overflow: visible !important;  
}
```

```
/* Ensure labels don't "stutter" during the move */
```

```
.leaflet-label {  
  will-change: transform, top, left;  
  transition: none !important;  
}
```

```
/* Ensure the path (polygon) doesn't disappear or flicker */
```

```
/* Keep the polygon fill stable */
```

```
path.leaflet-interactive {  
  /* Prevents borders from getting thick and overlapping during zoom */  
  vector-effect: non-scaling-stroke !important;  
  stroke-width: 1px !important;
```

```
}

</style>

<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.3/dist/leaflet.css" />

<script src="https://unpkg.com/leaflet@1.9.3/dist/leaflet.js"></script>

    <script src="https://unpkg.com/leaflet@1.9.3/dist/leaflet.js"></script>

<link rel="stylesheet" href="./leaflet-mapwithlabels.css" />

<style type="text/css">


</style>

<link rel="stylesheet" href="https://cdnjs.cloudflare.com" />

<link href='https://cdnjs.cloudflare.com/ajax/libs/leaflet/0.7.7/leaflet.css'

    rel='stylesheet' type='text/css'/>

<meta http-equiv="X-UA-Compatible" content="IE=Edge">


</head>

<body>

    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

<script src="https://d3js.org"></script>

<script src="./leaflet.markercluster-src.js"></script>

<script src="https://unpkg.com/leaflet.markercluster.layersupport@2.0.1/dist/
leaflet.markercluster.layersupport.js"></script>

<script src="L.D3SvgOverlay.min.js"></script>
```

<script>

```
// 1. Initialize Map
```

```
var map;
```

```
try {
```

```
  map = L.mapWithLabels('map-canvas', {
```

```
    center: [42.3840352, -71.1134377],
```

```
    zoom: 13
```

```
  });
```

```
} catch (e) {
```

```
  console.warn("mapWithLabels plugin not found, using standard Leaflet map.");
```

```
  map = L.map('map-canvas').setView([42.3840352, -71.1134377], 13);
```

```
}
```

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
```

```
  attribution: '&copy; OpenStreetMap contributors'
```

```
}).addTo(map);
```

```
var neighborhoodData = [];
```

```
// 2. Define the D3 Overlay (Shapes + Labels + Interactions)
```

```
var d3Overlay = L.d3SvgOverlay(function (sel, proj) {
```

```
  // --- DRAW NEIGHBORHOOD SHAPES ---
```

```
  var paths = sel.selectAll('path').data(neighborhoodData);
```

```

paths.enter()

  .append('path')

  .attr('class', 'neighborhood')

  .attr('stroke', 'black')

  .attr('fill', function() { return d3.hsl(Math.random() * 360, 0.7, 0.5); })

  .attr('fill-opacity', '0.5')

// HOVER EFFECTS

.on('mouseover', function(d) {

  d3.select(this)

    .transition().duration(150)

    .attr('fill-opacity', '0.8')

    .attr('stroke-width', 3 / proj.scale);

})

.on('mouseout', function(d) {

  d3.select(this)

    .transition().duration(150)

    .attr('fill-opacity', '0.5')

    .attr('stroke-width', 1 / proj.scale);

})

// CLICK TO ZOOM

.on('click', function(d) {

  map.fitBounds(L.geoJson(d).getBounds());

});

// Sync path projection to map zoom/pan

paths.attr('d', proj.pathFromGeojson)

  .attr('stroke-width', 1 / proj.scale);

```

```
// --- DRAW LABELS ---
```

```
var labels = sel.selectAll('text').data(neighborhoodData);
```

```
labels.enter()
```

```
  .append('text')
```

```
  .style('fill', 'black')
```

```
  .style('font-weight', 'bold')
```

```
  .style('text-anchor', 'middle')
```

```
  .style('text-shadow', '1px 1px 1px white, -1px -1px 1px white')
```

```
  .style('pointer-events', 'none') // Critical so clicks hit the paths below
```

```
  .text(function(d) { return d.properties.NAME || d.properties.name || ""; });
```

```
labels.each(function(d) {
```

```
  // Find center of each polygon for label placement
```

```
  var center = L.geoJson(d).getBounds().getCenter();
```

```
  var point = proj.latLngToLayerPoint(center);
```

```
  d3.select(this)
```

```
    .attr('x', point.x)
```

```
    .attr('y', point.y)
```

```
    .style('font-size', (14 / proj.scale) + 'px');
```

```
});
```

```
});
```

```
// 3. LOAD DATA AND ACTIVATE
```

```
// IMPORTANT: Ensure this filename matches your .geojson file exactly!
```

```
d3.json("./cambridge_somerville_neighborhoods.geojson", function(error, data) {
```

```
  if (error) {
```

```
    console.error("GeoJSON failed to load. Check file path or use a local server.", error);  
    return;  
}
```

```
neighborhoodData = data.features;
```

```
// Add the D3 layer to the map now that we have data
```

```
d3Overlay.addTo(map);
```

```
console.log("Map initialized with", neighborhoodData.length, "neighborhoods.");
```

```
});
```

```
var mcgLayerSupportGroup = L.markerClusterGroup.layerSupport();
```

```
// Create separate layer groups for different types of markers
```

```
var stores = L.layerGroup();
```

```
var homes = L.layerGroup();
```

```
var bookstores = L.layerGroup();
```

```
// Check these layer groups into the MCG Layer Support
```

```
mcgLayerSupportGroup.checkIn([stores, homes, bookstores]);
```

```
// Add markers to their respective layer groups
```

```
L.marker([42.38985, -71.1185]).bindPopup('StarMarket').addTo(stores);
```

```
L.marker([42.39392, -71.125408]).bindPopup('Pemberton').addTo(stores);
```

```
L.marker([42.367843, -71.099929]).bindPopup('229 Harvard').addTo(homes);
```

```
L.marker([42.401424, -71.13354]).bindPopup('26 Murray Hill Road').addTo(homes);
```



```
L.marker([42.3817724,-71.0872455]).bindPopup('All She Wrote Books').addTo(bookstores);
```

```
L.marker([42.3812783,-71.1008173]).bindPopup('Side Quest Books &  
Games').addTo(bookstores);
```

```
L.marker([42.372541, -71.116363]).bindPopup('Harvard Book Store').addTo(bookstores);
```

```
L.marker([42.3873306,-71.1214895]).bindPopup('Porter Square Books').addTo(bookstores);
```

```
L.marker([42.3812973,-73.5779292]).bindPopup('Bryn Mawr Bookstore').addTo(bookstores);
```

```
L.marker([42.3738631,-71.1242451]).bindPopup('Lovestruck Books').addTo(bookstores);
```

```
mcgLayerSupportGroup.addTo(map);
```

```
// Add the layer groups to the map to display them (they will be clustered automatically)
```

```
stores.addTo(map);
```

```
homes.addTo(map);
```

```
bookstores.addTo(map);
```

```
// Add the standard Leaflet layers control
```

```
var overlays = {
```

```
  "Grocery Stores": stores,
```

```
  "Homes": homes,
```

```
  "Bookstores": bookstores
```

```
};
```

```
L.control.layers(null, overlays).addTo(map);
```

```
var markers = L.markerClusterGroup({spiderfyOnMaxZoom: false, showCoverageOnHover: false,  
zoomToBoundsOnClick: false});
```

```
markers.on('clusterclick', function (a) {
```

```
  a.layer.zoomToBounds({padding: [60, 60]});
```

```
});
```

```
// Store the layer so we can find Davis Square later

const neighborhoodLayer = L.geoJSON(somervilleNeighborhoods, {
  label: l => l.feature.properties.NAME,
  labelPos: 'cc',
  labelStyle: { fontWeight: 'bold', whiteSpace: 'normal', minWidth: '120px', textAlign: 'center' },
  labelPriority: 5e4,
  style: polyColor
}).addTo(map).bindPopup(l => ` ${l.feature.properties.NAME} `);
```

```
// 2. THE HIGH-PRIORITY SYNC LISTENER
```

```
window.addEventListener('message', function(event) {
  if (event.data.action === 'sync-resize-fly') {

    let targetCenter;

    neighborhoodLayer.eachLayer(layer => {
      if (layer.feature.properties && layer.feature.properties.NAME === "Davis Square") {
        targetCenter = layer.getBounds().getCenter();
      }
    });

    if (targetCenter) {

      // 1. Immediately force a re-calculation
      map.invalidateSize({ animate: false, pan: false });

      // 2. Start the flyTo
      map.flyTo(targetCenter, 16, {
```

```
animate: true,  
duration: 1.0,  
noMoveStart: true, // Prevents vectors from disappearing or jumping at the start  
easeLinearity: 0.1  
});
```

```
    // 3. THE "HAMMER" LOOP: Force redraws every 50ms for 1 second  
    // This is the most reliable way to fix the "clumped in corner" bug.  
    let elapsed = 0;  
    const syncTimer = setInterval(() => {  
        map.invalidateSize({ animate: false, pan: false });  
  
        // This specific line forces the SVG polygons to stretch  
        if (map._renderer) {  
            map._renderer._update();  
        }  
  
        // Keeps labels pinned to the polygons  
        if (typeof map._updateLabels === 'function') {  
            map._updateLabels();  
        }  
  
        elapsed += 50;  
        if (elapsed > 1000) clearInterval(syncTimer);  
    }, 50);  
}  
}  
});
```

</script>

</body>

</html>